

TEKNISK RAPPORT

ÖVER UTFÖRT ARBETE

Just Energy API

<https://github.com/Stockholm-3/just-api>

<https://github.com/Stockholm-3/lib>

<https://github.com/Stockholm-3/just-client>

HTTP-server för väderdatahantering och energiplanering

Lärosäte	Chas Academy
Utbildningsprogram	SUVX25
Team	Team Stockholm 3
Startdatum	2025-11-04
Implementeringsspråk	C (standard C11 / GNU), C++11
Licens	MIT License
Externa API:er	Open-Meteo (väder), Elpris SE (elpriser)

1. Allmän projektbeskrivning

Just API är en fullständig HTTP-server, utvecklad i programmeringsspråket C inom ramen för ett skolprojekt på Chas Academy (program SUVX25). Systemet fungerar som ett mellanliggande lager (middleware) mellan klientapplikationer och externa datakällor: vädertjänsten Open-Meteo och den svenska elpriskällan Elpris.

Projektets kärnvärde ligger i att kombinera två informationsflöden — meteorologiska data och aktuella marknadspriser på el — för att skapa en "intelligent" 24-timmarsplan för energianvändning i hushåll utrustade med solpaneler och batterier.

1.1. Projektmål och uppgifter

- Implementera en egen HTTP-server utan att använda färdiga webbramverk
- Proxying och cachning av data från externa REST API:er
- Utveckla en algoritm för optimering av elförbrukning
- Bygga en tillförlitlig serverinfrastruktur med automatisk återstartningsmekanism (watchdog)
- Stöd för konkurrerande requesthantering via en trådpool

1.2. Teknikstack

Komponent	Teknik / Bibliotek	Syfte
Huvudspråk	C11 (GNU extension)	Implementering av server, daemon och algoritm
Hjälpsspråk	C++11	Thread pool
JSON-parsning	Jansson (inbyggd)	Serialisering / deserialisering av API-svar
Hashing	MD5 (egen impl.)	Generering av filcachenyckar
Byggsystem	GNU Make	Kompilering och beroendehantering
Containerisering	Docker	Driftsättning i produktionsmiljö
Dokumentation	Doxygen	Automatisk generering av API-dokumentation

2. Systemarkitektur

Projektets arkitektur är uppbyggd på principen om tydlig ansvarsfördelning. Systemet består av fyra huvudprocesser/tråd som kommunicerar med varandra via definierade gränssnitt. (ofta genom energy plan filhantering).

2.1. Processöversikt

Process / Komponent	Binärfil/tråd	Roll
Watchdog Daemon	watchdog (src/bin/watchdog)	Övervakning och automatisk omstart av servern
HTTP Server	server (src/bin/server)	Hantering av klientförfrågningar
Compute Engine	compute (src/bin/compute)	Körning av energiplaneringsalgoritmen
Fetch Scheduler	(tråd i watchdog) (src/watchdog/fetch_scheduler)	Regelbunden hämtning av väder- och prisdata

2.2. Watchdog Daemon

Systemets ingångspunkt är watchdog — en daemon implementerad i filen main.c. Dess ansvarsområden inkluderar:

- Tolkning av kommandoradsargument (getopt_long)
- Initiering av delsystem: logger, lagring av energiplaner, hämtningsschemaläggare
- Valfri daemonisering av processen (dubbel fork, setsid, omdirigering av standarddeskriptorer)
- Skrivning av PID-fil och konfigurering av signalhanterare (SIGTERM, SIGCHLD)
- Huvudövervakningsloop: spårning av barnprocessens tillstånd och omstart vid kraschar

Watchdog använder modulen restart_policy för att implementera exponentiell backoff vid upprepade fel med konfigurerbara parametrar:

```
initial_backoff_ms: 1000    // 1 sekund
max_backoff_ms:      30000  // 30 sekunder
max_restarts:        10     // inom restart_window_sec
restart_window_sec:  60
```

2.3. HTTP-server och anslutningshanteringsmodell

HTTP-servern är byggd ovanpå en egen TCP-serverimplementering. Arkitekturen använder en händelsedrivna anslutningsmodell baserad på modulen **SMW (State Machine Worker)**, som ger icke-blockerande I/O-hantering.

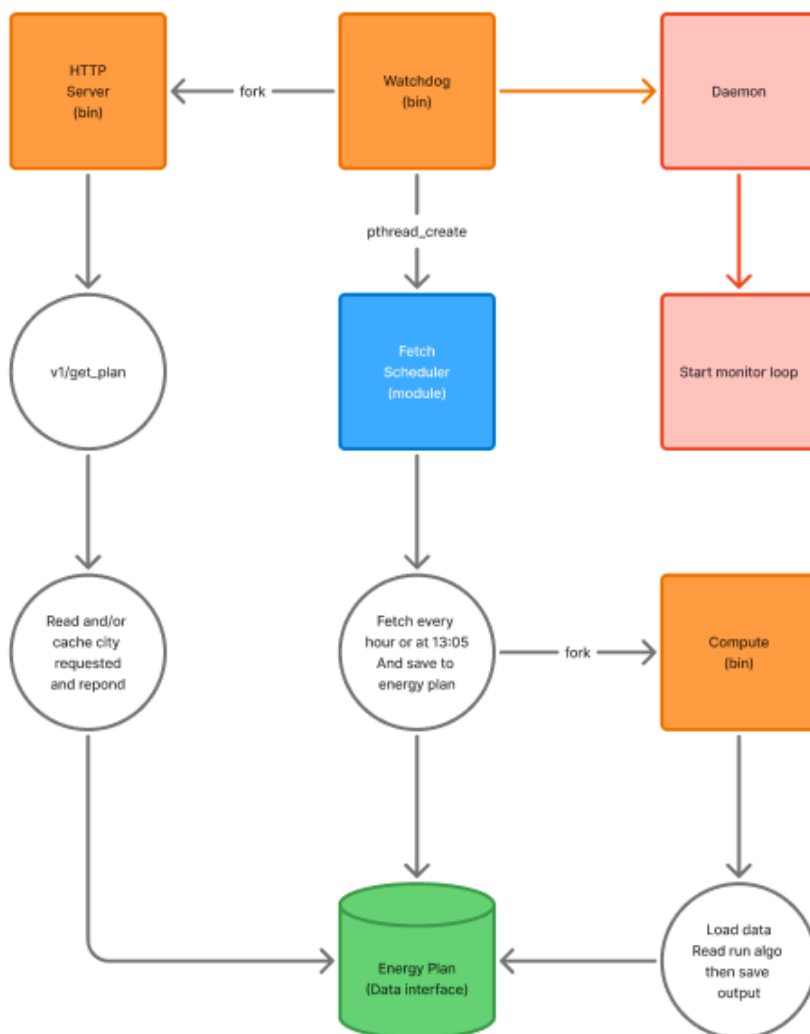
Hanteringskedja för inkommande anslutning:

- TCPServer accepterar anslutningen (accept)
- HTTPServer skapar ett HTTPServerConnection-objekt för klienten
- WeatherServer registrerar anslutningen i en länkad lista (LinkedList)
- WeatherServerInstance läser HTTP-förfrågan och dirigerar till rätt hanterare
- ThreadPool utför blockerande operationer (HTTP-anrop till API, filläsning) i bakgrundstrådar
- Resultatet returneras via callback-mekanismen till huvud-eventloopen

2.4. Konfiguration (config.json)

All systemkonfiguration är samlad i filen config.json och laddas av modulen config_parser. Nyckelparametrar:

Sektion	Parameter	Standardvärde	Beskrivning
server	port	10680	TCP-port för servern
server	max_connections	1024	Max antal samtidiga anslutningar
cache	weather_ttl_seconds	900	Väder-cache TTL (15 min)
cache	geo_ttl_seconds	604800	Geokodnings-TTL (7 dagar)
thread_pool	num_workers	4	Antal arbetstrådar
thread_pool	max_pending	256	Uppgiftskö
watchdog	max_restarts	10	Omstartsgräns
energy_plan	max_cities	200	Max antal spårade städer



3. Beskrivning av nyckelmoduler

3.1. Energy Plan (energy_plan_store) (src/energy_plan)

Energy plan store är ett interface som hanterar filläsning, filskrivning, fil låsning och cache-logik med allt som handlar om energy plan. Energy plans sparas under folder `./energy_plan`. så det är som en liten databas bara för energiplanen. För att förhindra race conditions vid skrivning och läsning av resultat används POSIX flock-fillåsning via modulen `file_lock`. Läsoperationer utförs asynkront via SMW-uppgifter, skrivning sker i bakgrundstrådar.

Filstruktur:

```
energy_plan/
├── cities.csv          # vilka städer som ska hämtas vid nästa fetch
├── compute_input/     #input till compute steg. (hämtat av fetch scheduler
│   ├── <stad>-<lat>-<lon>.json  # Väderdata för staden
│   └── elpris_merged.json      # Sammanslagen prisdata
└── compute_output/    #genererad output från compute
    ├── .lock           # Fillås (flock)
    └── <Stad>-<Zon>.json  # Algoritmresultat
```

3.2. Hämtningsschemaläggare (fetch_scheduler)

FetchScheduler körs i en separat pthread-tråd och hanterar två oberoende datahämtningsscheman:

- **Väderdata** — hämtas varje timme (`weather_interval_ms`: 3 600 000 ms) med en förskjutning på +2 minuter från timmes start (`weather_offset_ms`: 120 000 ms). Data begärs för alla registrerade städer parallellt.
- **Elpriser** — hämtas en gång per dygn kl. 13:05 UTC (`elpris_hour_utc`: 13, `elpris_minute_utc`: 5) för fyra svenska priszoner: SE1, SE2, SE3, SE4.

Efter varje lyckad hämtning startar schemaläggaren (`fork/exec`) den binära filen `compute` för omberäkning av energiplaner.

3.3. Trådpool (thread_pool.cpp)

Trådpoolen är implementerad i C++11 och tillhandahåller en trådsäker uppgiftskö för att köra blockerande operationer utanför huvud-event loopen. Implementeringsfunktioner:

- Stöd för uppgiftsprioritet och körningstimeout
- Kooperativ avbokning av uppgifter (`thread_pool_task_is_cancelled`)
- Aviseringsmekanism via pipe (epoll-kompatibel) för pollningsfri integration med event loopen
- Callback-modell: `done_fn` körs i huvudtråden efter att bakgrunds arbetet slutförts
- Statistikinsamling: aktiva trådar, kö, antal slutförda uppgifter

4. Energioptimeringsalgoritm (src/energy_plan/algo.c)

Kärnan i projektet är energioptimeringsalgoritmen implementerad i modulen algo.c. Den analyserar meteorologiska data och prisdata, och genererar rekommendationer för varje giltigt 15-minuters intervall under dygnet.

4.1. Indata- och utdatastrukturer

Algoritmen arbetar med följande strukturer:

```
typedef struct {  
    double elpris[96];           // Elpris (96 slot × 15 min = 24 tim)  
    double temperature[96];     // Lufttemperatur  
    double sun_intensity[96];   // Solstrålningsintensitet  
    int slots;                  // Antal giltiga slot (0 = alla 96)  
} AlgoInput;
```

```
typedef struct {  
    double buy_electricity;      // Andel: köp från nätet  
    double direct_use;          // Andel: direkt förbrukning  
    double charge_battery;      // Andel: ladda batteriet  
    double sell_excess;         // Andel: sälj överskott  
} AlgoSlot; // Summan av fyra värden = 1,0 för varje slot
```

4.2. Beslutslogik

För varje giltig tidslucka tar algoritmen beslut baserat på tre primära faktorer:

- **Elpris:** avgör lämpligheten att köpa eller sälja till nätet
- **Lufttemperatur:** styr fördelningen mellan direkt användning och försäljning i högprisfallet
- **Solintensitet:** styr fördelningen mellan batteriladdning och köp från nätet i lågprisfallet
- **Radiation:** används inte som separat beslutsfaktor i algoritmsteget, utan normaliseras först till solintensitet (0-1)

Algoritmen använder dygnets medelpris som referens och beräknar rekommenderad fördelning i fyra kategorier. Det slutliga AlgoOutput-resultatet innehåller rekommendationer för varje tidslucka samt ett sammanfattande medelvärde (summary) för dygnet.

Temperatur och solintensitet används som fördelningsfaktorer inom två olika prisregimer:

- **Lågprisfall ($\text{price_delta} \leq 0$):** solintensitet avgör hur stor del av den aktiva andelen som går till batteriladdning jämfört med köp från nätet. Högre solintensitet ger större andel till charge_battery, medan lägre solintensitet ger större andel till buy_electricity
- **Högprisfall ($\text{price_delta} > 0$):** temperatur avgör hur stor del av den aktiva andelen som går till direkt användning jämfört med försäljning. Högre temperatur ger större andel till direct_use (via temperatur/40, begränsad till 0–1), medan lägre temperatur ger större andel till sell_excess.

4.3. Prisreferens och viktning

Algoritmen beräknar först ett medelpris för dygnet och jämför varje tidsluckas pris med detta referensvärde. Prisavvikelsen skalas med ett band ($0,5 \times \text{medelpris}$, eller 1,0 om medelpriset är ≤ 0) och omvandlas därefter till en beslutsvikt som begränsas till intervallet -1 till 1. Större avvikelse från medelpriset ger starkare rekommendation.

4.4. Fördelning per tidslucka och dygnsammanfattning

Vid lågpris fördelas den aktiva andelen mellan köp från nätet och batteriladdning, där solintensitet styr fördelningen; återstående andel går till sell_excess. Vid högpris fördelas den aktiva andelen mellan direkt användning och försäljning, där temperatur styr fördelningen; återstående andel går till

buy_electricity. För varje tidslucka summerar andelarna till 1,0. Därefter beräknas summary som medelvärdet per kategori över alla giltiga tidsluckor.

5. REST API — Endpointbeskrivning

Servern tillhandahåller ett REST API på port 10680. Alla svar returneras i JSON-format med lämpliga Content-Type HTTP-huvuden.

5.1. Väder-endpoints

Metod	Sökväg	Parametrar	Beskrivning
GET	/v1/current	lat, lon	Aktuellt väder via koordinater
GET	/v1/hourly	lat & lon / city	Timvädersprognos
GET	/v1/minutely	lat & lon / city	Minutprognos (för planering)
GET	/v1/weather	city	Väder via stadsnamn (geokodning)

5.2. Elpris- och planeringsendpoints

Metod	Sökväg	Parametrar	Beskrivning
GET	/v1/elpris	price (SE1–SE4)	Aktuella elpriser
GET	/v1/get_plan	city, price (SE1–SE4)	24-timmarsplan för energioptimering
GET	/v1/cities	—	Lista över registrerade städer

5.3. Serviceendpoints

Metod	Sökväg	Beskrivning
GET	/health	Serverstatuskörning (Health check)
GET	/echo	Diagnostisk eko av förfrågan
GET	/	Startsida (index.html)

6. Tillförlitlighet och feltolerans

6.1. Watchdog och omstartspolicy

Systemet är utformat för kontinuerlig drift. Watchdog-daemonen övervakar HTTP-serverns tillstånd via waitpid med nollskilt exitstatus och initierar omstart vid krasch. Innan omstart utförs en tillgänglighetskontroll av servern via HTTP health check.

Mekanismen för exponentiell backoff förhindrar cyklisk omstart vid systematiska fel: fördröjningen mellan försök fördubblas från 1 s till 30 s. Om antalet omstarter överstiger 10 per minut avslutas watchdog med fel.

6.2. Signalhantering

Systemet hanterar Unix-signaler korrekt:

- **SIGTERM**: initierar graceful shutdown — väntar på att alla trådar avslutas och anslutningar stängs
- **SIGCHLD**: meddelar watchdog om barnprocessens avslutning
- **SIGPIPE**: ignoreras (skydd mot skrivning till stängda socketar)

6.3. Fillåsning

Vid skrivning av algoritmresultat används POSIX flock för exklusiv fillåsning. Detta garanterar atomicitet för operationer mellan watchdog- och compute-processerna, även vid parallell körning.

7. Bygge och driftsättning

7.1. Byggsystem (Makefile)

Projektet använder GNU Make med stöd för inkrementell kompilering. Huvudsakliga mål:

```
make daemon-start    # Kompilering och start av watchdog i bakgrunden
make daemon-stop     # Stopp av watchdog (SIGTERM)
make docs            # Generering av Doxygen-dokumentation
```

7.2. Docker-container

För produktionsdriftsättning har en Dockerfile med multi-stage build förberetts. Entrypoint-skriptet (entrypoint.sh) säkerställer korrekt initiering av filsystemet och start av watchdog i förgrundsläge i containermiljön.

7.3. Beroendestruktur

Projektet använder git submodule för att hantera externa bibliotek via det gemensamma förrådet stockholm-3-lib:

- Jansson — JSON-parsning (byggs från källkod)
- Http client med MbedTLS
- Andra självskrivna moduler och helpers som är nödvändiga för projektet

8. Kodkvalitet och utvecklingsstandarder

8.1. Kodstil

Projektet följer en enhetlig stil som upprätthålls av verktyg för statisk analys och formatering:

- **clang-format:** automatisk kodformatering enligt .clang-format
- **clang-tidy:** statisk analys nuvarande bara för kod standard(.clang-tidy)
- **Doxygen:** dokumentering av publika API:er via kommentarer i Doxygen-format (Doxyfile)

8.2. Profileringsinfrastruktur

I projektet finns en fullständig profileringsinfrastruktur implementerad via tre separata Makefile-mål.

- **GNU gprof (make profile)**

Kompilerar binärfilen med flaggan -pg (utan att ändra optimeringsnivån -O1), kör den och genererar automatiskt en textrapport via gprof. Resultatet sparas i gprof.txt (konfigurerbart via variabeln GPROF_OUT). Parametern TIMEOUT stöds för kontrollerad terminering av servern innan analys. Artefakten gmon.out (83 KB) finns i repot.

Begränsning: gprof visar bara tid i funktioner i huvudtråden; arbetsbelastningen i bakgrundstrådar (thread pool) återspeglas inte korrekt på grund av hur POSIX-trådar samplas.

- **Valgrind Callgrind (make callgrind)**

Kör compute under valgrind --tool=callgrind och sparar anropsgraf i callgrind.out (209 KB).

Resultatet är avsett för visualisering i KCachegrind eller analys via callgrind_annotate. Callgrind ger ett exakt anropstråd utan sampling — varje funktionsanrop instrumenteras, vilket gör det möjligt att se det verkliga antalet instruktioner och cachemissar.

Begränsning: Callgrind-overhead saktar ner körningen 10–50×, så profilering utförs under begränsad belastning.

- **Valgrind Memcheck (make valgrind / make memcheck)**

Kontrollerar minnesläckor med flaggorna --leak-check=full --show-leak-kinds=all --track-origins=yes. make valgrind kör alla tre binärfiler (watchdog, server, compute) sekventiellt. make memcheck kör en enskild binärfil med exitkod 1 vid definitiva läckor — integrerbart i CI.

8.3. Utvecklingsprinciper

- Tydlig ansvarsfördelning: varje modul har ett enda syfte
- Early return-mönster: undvikande av djup villkorsnästning
- Explicit minneshantering: läckfri design (leak-free)
- Callback-arkitektur för asynkrona operationer utan att blockera eventloopen
- Trådsäkerhet: alla delade resurser skyddas av mutex eller atomvariabler

9. Resultat och slutsatser

9.1. Uppnådda resultat

Inom ramen för skolprojektet har Team Stockholm 3 framgångsrikt implementerat:

- En fullständig HTTP REST API-server i C utan webbramverk
- Egen nätverksstackTCP-server → HTTP-hanterare → requestdirigering
- Tvånivåcachningssystem (in-memory + filcache) med TTL-kontroll
- Watchdog-daemon med exponentiell backoff och graceful shutdown
- Integration med två externa API:er via HTTPS (Open-Meteo, Elpris SE)
- Algoritm för 24-timmarsplanering av energiförbrukning baserad på 96 tidsluckor
- Schemaläggare för regelbunden datahämtning med anpassat schema
- Trådsäker trådpool (C++11) för blockerande operationer
- Docker-containerisering för produktionsdriftsättning

9.2. Tekniska systemprestanda

Mätvärde	Värde
Max antal samtidiga anslutningar	1 024
Antal arbetstrådar	4 (konfigurerbart)
Väder-cache TTL	15 minuter
Geokodnings-TTL	7 dagar
Max antal spårade städer	200
Väderuppdateringsfrekvens	Varje timme (HH:02:00 UTC)
Prisuppdateringsfrekvens	En gång per dygn (13:05 UTC)
Tidsluckor i planen	96 (15-minutersintervall)
Stödda priszoner	SE1, SE2, SE3, SE4
Omstartsgrens	10 per 60 sekunder

9.3. Slutsats

Projektet Just Weather API är en praktisk demonstration av programmeringsspråket C:s möjligheter för att bygga produktionsklara serversystem. De implementerade lösningarna täcker ett brett spektrum av systemprogrammering: nätverks-I/O, POSIX-trådar, IPC, processhantering, filsystem och algoritmisk optimering.

Problem	Modul	Status
LinkedList $O(n)$ för anslutningar och cache	<code>weather_server.c</code> , <code>cache.c</code>	Känt, accepterat
Linjär router (14 rutter)	<code>routes.h</code>	Ej kritiskt
Saknad retry för externa API:er	<code>fetch_scheduler.c</code> , <code>http_client.c</code>	Kräver vidareutveckling

LinkedList för aktiva anslutningar — linjär sökning

Problem

`WeatherServer` lagrar aktiva `WeatherServerInstance`-objekt i en `LinkedList*`. Borttagning av en avslutad anslutning kräver en linjär genomgång av listan. Med konfigurationen `max_connections: 1024` innebär det i värsta fall 1 024 jämförelser per anslutningsstängning. På samma sätt implementerar `cache.c` in-memory-cache med `LinkedList`, vilket gör varje sökning till $O(n)$.

Lösning

För lagring av anslutningar är det lämpligt att använda en hashtabell med socketbeskrivaren som nyckel. Det reducerar kostnaden för sökning och borttagning till amorterat $O(1)$. För cachan gäller detsamma: en hashtabell med strängnycklar (på samma sätt som i `Cache.cpp`, men `cache.c` bör anpassas till samma mönster).

Varför det gjordes så här

En länkad lista är den enklaste dynamiska datastrukturen att implementera från grunden i C. För ett skolprojekt med ett litet antal samtidiga anslutningar märks skillnaden mellan $O(n)$ och $O(1)$ inte. Samtidigt demonstrerar det förståelse för grundläggande datastrukturer, vilket i sig är ett lärandemål.

Linjär genomgång av routingstabellen (`routes.h`)

Problem

Routern i `weather_server_instance_on_request()` itererar linjärt genom arrayen `g_routes[]` (14 rutter) med `strcmp` på metod och sökväg för varje förfrågan. Trots att 14 element är ett litet antal skalerar ansatsen inte: varje ny endpoint ökar den genomsnittliga sökkostnaden.

Lösning

Implementera ett prefixträd (trie) eller en hashtabell för rutter med nyckeln `method + path`. Det ger $O(k)$ -sökning (där k är sökvägens längd) oberoende av antalet rutter. För nuvarande skala skulle till och med en enkel `switch` på sökvägens första tecken ge märkbar förbättring.

Varför det gjordes så här

Linjär sökning är den enklaste tänkbara implementeringen av en routningstabell. Med ett fast litet antal endpoints (14 st.) är prestandaskillnaden jämfört med ett trie försumbar i relation till den totala behandlingstiden per förfrågan, som domineras av I/O-operationer. Läsbarheten i koden är dessutom maximal.

Saknad retry-mekanism för externa API-anrop

Problem

Anrop till Open-Meteo och Elpris API i `http_client.c` och `tls_client.c` saknar en inbyggd mekanism för upprepade försök. Om ett externt API är tillfälligt otillgängligt eller returnerar 5xx avslutas klientens förfrågan med ett fel omedelbart. FetchScheduler väntar vid ett misslyckat försök helt enkelt till nästa schemalagda körning (upp till 1 timme för väderdata).

Lösning

Implementera exponential backoff retry direkt i `fetch_scheduler.c`: Vid fel från API:et utförs upp till 3 nya försök med fördröjningarna 5 s → 15 s → 45 s innan felet returneras. Separat — en circuit breaker för att förhindra lavinliknande upprepade förfrågningar vid längre driftstopp hos den externa tjänsten.

Varför det gjordes så här

Exponential backoff är redan implementerat i `restart_policy.c` för watchdog — den konceptuella förståelsen finns alltså. Att överföra denna mekanism till HTTP-klientnivå var inte en prioritet inom ramen för uppgiften. För en demonstrationstjänst där Open-Meteo normalt sett är tillgängligt är avsaknaden av retry ingen kritisk brist.

Förbättringsområden för framtida iterationer:

- Enhetstester — testinfrastrukturen är minimal. En övergång till Unity-testramverket skulle ge strukturerad täckning för kritiska moduler.
- Teknisk skuld i äldre moduler — dessa moduler skrevs tidigt i projektet och har aldrig omarbetats. De speglar inte de mönster och lärdomar som vi har samlat på oss under projektets gång.
- Kvarvarande lint-varningar — clang-tidy rapporterar varningar men blockerar inte bygget. Avsikten är att varningar successivt åtgärdas och att clang-tidy på sikt integreras som ett hårt bygghinder i CI-pipelinan.
- Hårdkodade konstanter — ett antal `#define`-värden i kodbasen (t.ex. `MAX_CLIENTS` i `tcp_server.h`) är inte kopplad till `config.json`. Dessa bör ersättas med konfigurationsinläsning för att ge operatören full kontroll utan omkompilering.